Forecasting Electricity Demand

A worked case study in time-series forecasting, with a real generation-mix leakage trap

AUTHOR
Tran Nguyen Vu

PUBLISHED
September 30, 2026

Interactive application

Open the Streamlit app

1 Introduction

This project predicts hourly electricity demand (`Consumption`, megawatts) for Romania's national grid, using published data from 2019 onward together with its generation mix (nuclear, wind, hydro, oil & gas, coal, solar, biomass). Demand is a **time series**: observations are ordered in time, so evaluation must preserve that order to avoid leaking future information into training.

The generation-mix breakdown makes it possible to test whether recent production patterns carry forecasting signal and to examine changes in Romania's renewable electricity production.

One thing worth flagging upfront: **this dataset is live and continuously updated** — checked directly, an earlier download had 46,002 rows through April 2024, the version used here has 62,810 rows through March 2026, with older rows unchanged. Every number below reflects one snapshot, not a fixed, versioned file.

2 Data

```
62810 rows, 2019-01-01 00:00:00 to 2026-03-14 23:00:00
```

2.1 Two real data-quality issues — and they are not the same issue

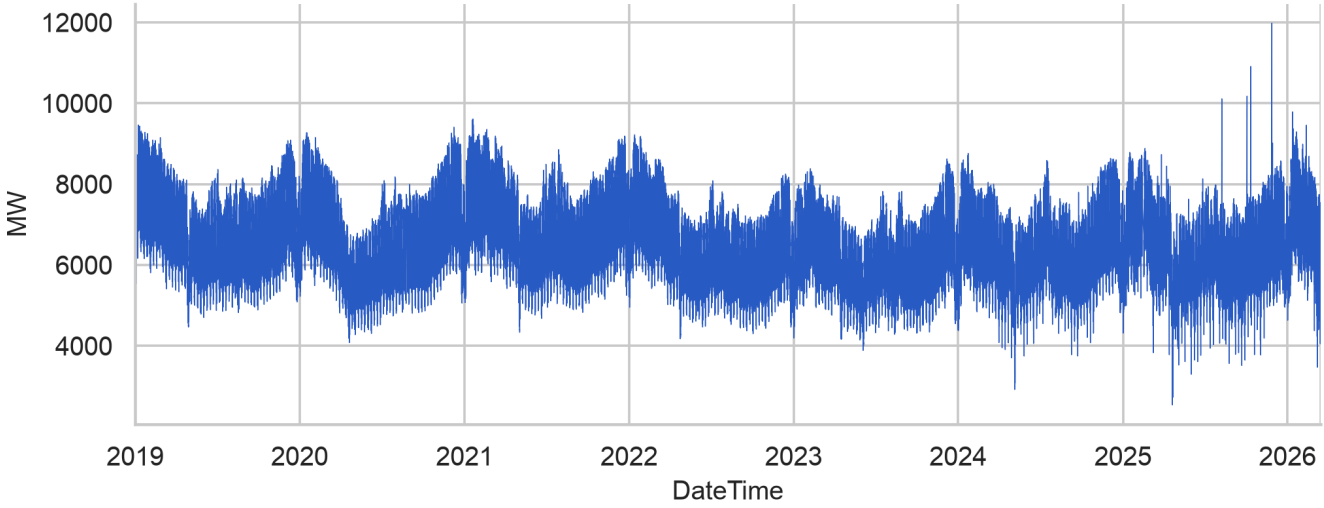
```
18 duplicate timestamps, 321 missing timestamps
78 separate gaps, longest: 83 hours
```

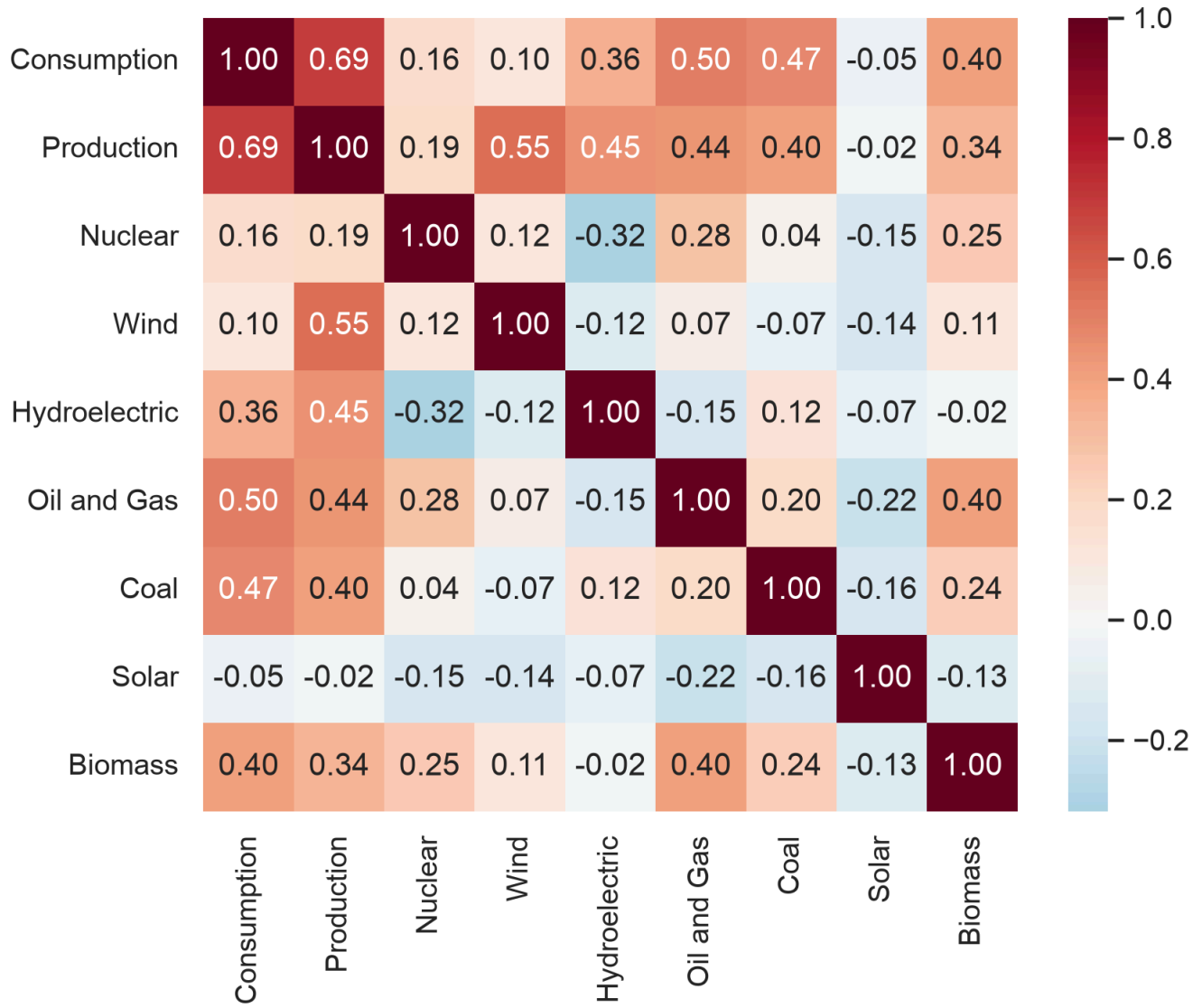
Every duplicate falls on an October date at **3am** local time, consistent with the Daylight Saving Time fall-back in Romania's Eastern European time zone. 2019-2023 show exactly 1 missing hour per year (the

spring-forward gap). **2024-2025 are a completely different story**: a single gap of **83 consecutive missing hours**, consistent with a real reporting/scraping outage in whatever live feed this dataset's maintainer pulls from — not a calendar quirk. I fixed both the same way (average duplicates, linearly interpolate gaps against a strictly regular hourly index), but want to be upfront that interpolating an 83-hour gap is a much rougher approximation than the isolated 1-hour DST gaps — there's no way to recover 83 hours of real demand swings from a straight line between two endpoints.

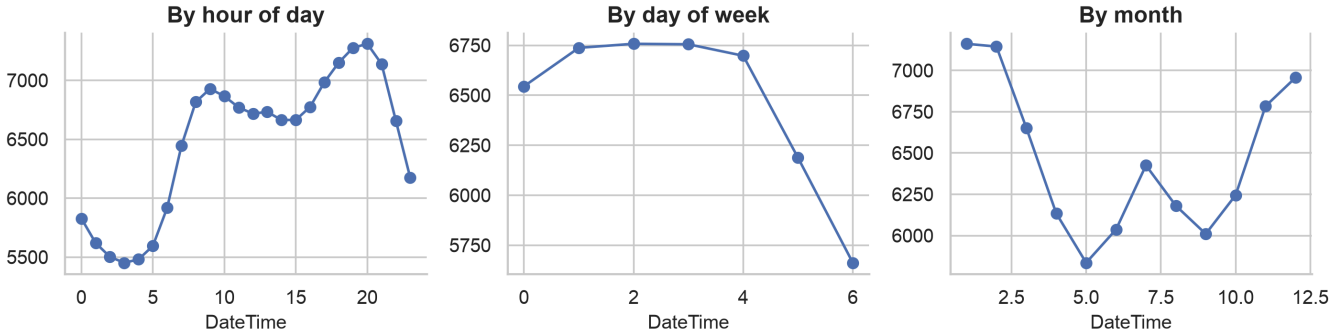
```
63120 hours, strictly regular: True
```

2.2 Exploring the series and its generation mix

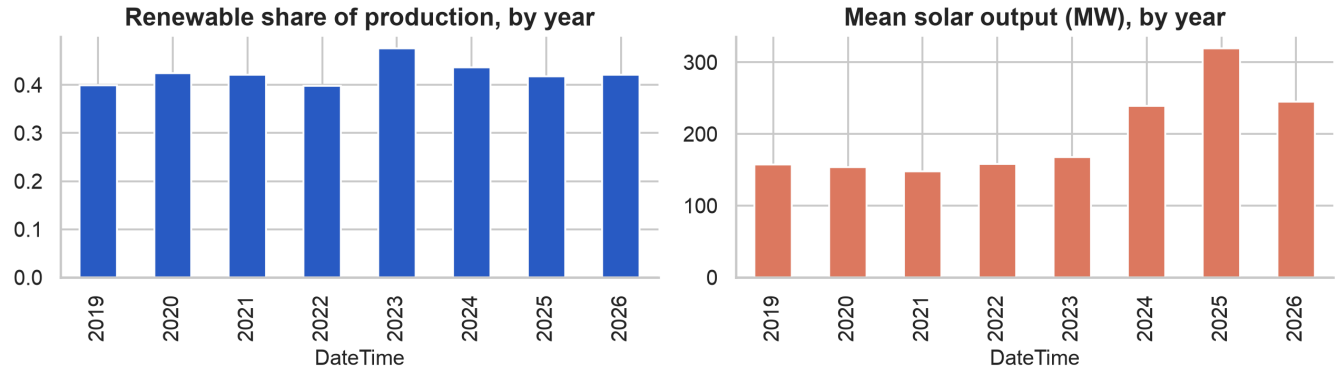




`Consumption` correlates most with `Production` (0.69, as expected) and the two dispatchable fossil sources, `Oil and Gas` (0.50) and `Coal` (0.47) — the grid operator leans on fossil generation specifically to follow demand swings. `Solar` is essentially uncorrelated with demand (-0.05), which is exactly what I'd expect: solar output follows daylight and weather, not consumption.



Demand has a single evening peak (~8-9pm, ~7,300 MW) and an overnight trough (~3-4am, ~5,450 MW). The monthly pattern is **winter-dominant**: demand peaks in January/February and troughs in May, with a smaller summer cooling peak.



The clearest "genuinely useful and interesting" finding from having the generation-mix breakdown at all: renewable share has held around 40-48% of production every year since 2019, but **mean solar output roughly doubled between 2023 (~168 MW) and 2025 (~319 MW)** — a real, checkable signature of Romania's recent solar buildout.

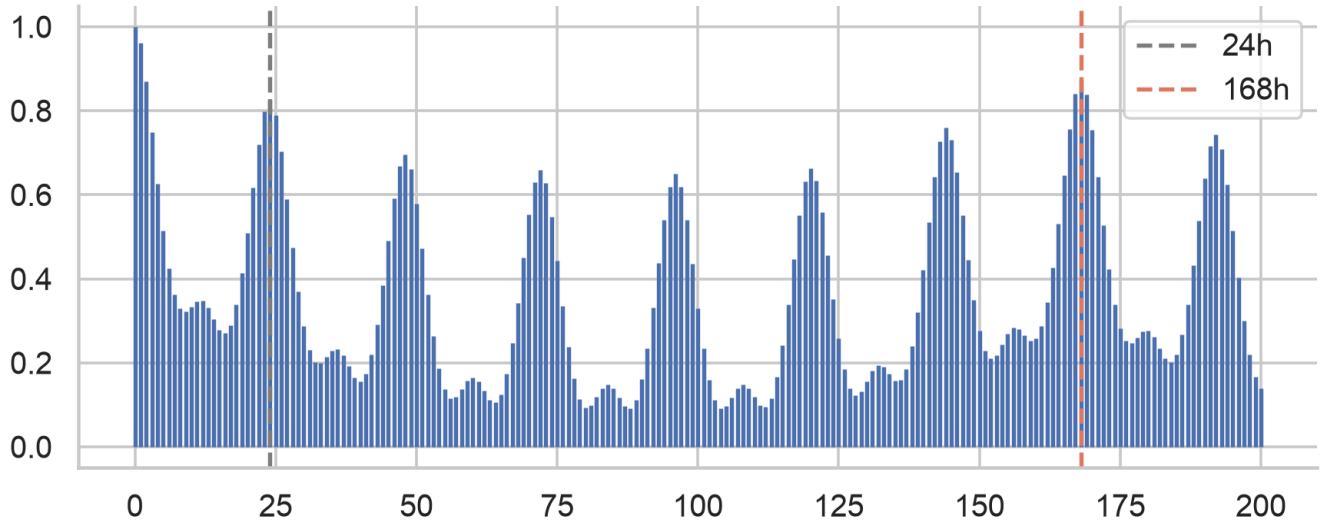
3 Statistical rigor before modeling

3.1 Is this series stationary?

```
ADF statistic: -14.38, p-value: 9.086e-27
```

Formally stationary (no unit root) — which looked wrong to me at first given the obvious daily/monthly patterns above. The resolution: ADF specifically tests for a *stochastic trend*, not for seasonality. A strong, bounded, repeating pattern always reverts toward its typical shape rather than wandering further from it, so it doesn't have a unit root even though it clearly isn't "flat."

3.2 Autocorrelation, used to justify lag choices empirically




```
ACF at lag 1h: 0.9614
ACF at lag 24h: 0.8267
ACF at lag 48h: 0.6955
ACF at lag 168h: 0.8730
```

Autocorrelation is genuinely high at both 24h (0.83) and 168h (0.87 — if anything slightly stronger), with a noticeably weaker mid-point at 48h (0.70) — empirical justification for these two specific lags, rather than picking them by convention.

4 Feature engineering

4.1 The leakage trap in the generation-mix columns

`Production` and its 7 source breakdowns are recorded at the **same hour** as `Consumption`, not before it. Using them directly would be circular for a real forecasting task, since generation is dispatched in response to demand, not fixed in advance. Every generation-mix feature is lagged by 24 hours.

DateTime	lag_24h	lag_168h	Solar_lag_24h	net_export_lag_24h	renewable_share_lag_24h
2026-03-14 21:00:00	7041.0	6402.0	0.0	-185.0	0.441219
2026-03-14 22:00:00	6426.0	6234.0	0.0	115.0	0.415839
2026-03-14 23:00:00	6019.0	5777.0	0.0	348.0	0.394220

4.2 A leakage-safe way to reuse a seasonal-decomposition idea

A natural next step is decomposing the series into trend/seasonal/ residual components and adding them as features. Doing this the most direct way — fitting `statsmodels`' decomposition on the whole series

before splitting — leaks: the trend component is a **centered** moving average using points both before *and after* each timestamp, so fitting it on the full series means training features are quietly built with future test-period information. `fit_seasonal_lookup` fits the decomposition **only on the training split** and reuses just the strictly periodic seasonal component (24 fixed hour-of-day offsets, safe to reapply to any timestamp without recomputation).

```
WITH seasonal_24h:    {'mae': 259.0764, 'rmse': 423.3834, 'mape': 0.0428}
WITHOUT seasonal_24h: {'mae': 258.6064, 'rmse': 424.0605, 'mape': 0.0428}
```

An honest negative result: adding the seasonal-decomposition feature makes essentially no difference (identical MAPE to 4 decimal places). LightGBM's tree splits likely already extract everything useful from the cyclical `hour_sin`/`hour_cos` encoding plus the lag/rolling features — worth reporting the idea that didn't help, not only the ones that did.

5 Methodology

5.1 Why the train/test split is chronological, not shuffled

Lag and rolling-window features are built from each row's own recent past, so a random shuffle would leak near-future information into training rows sitting next to shuffled-in test rows. The split is by position along the time-sorted index (last 15% of hours as test), and cross-validation uses `TimeSeriesSplit`, never `KFold`.

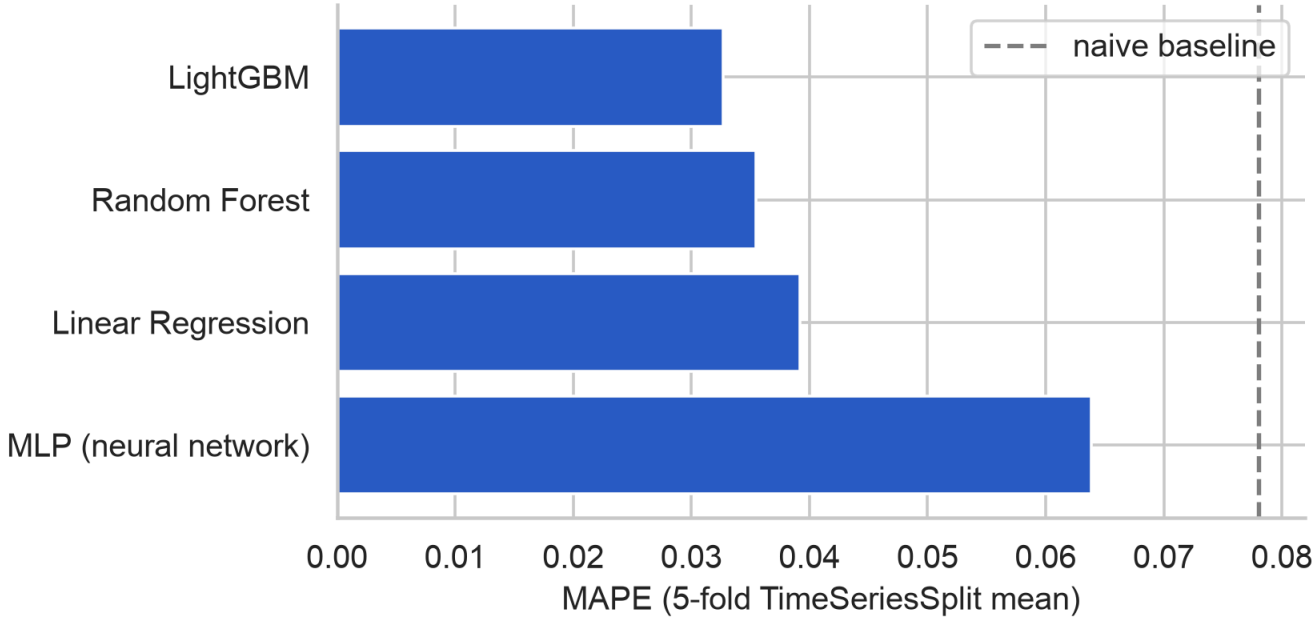
5.2 A naive baseline, measured before trusting any model

```
{'mae': 469.9865, 'rmse': 743.9772, 'mape': 0.078}
```

"Same hour last week" is off by **~7.8%** on average (MAPE) — note this test period includes some of the messy, interpolated 2024-2025 outage stretches, so this baseline is measured under harder conditions than 2019-2023 alone would give.

5.3 Model comparison, via TimeSeriesSplit

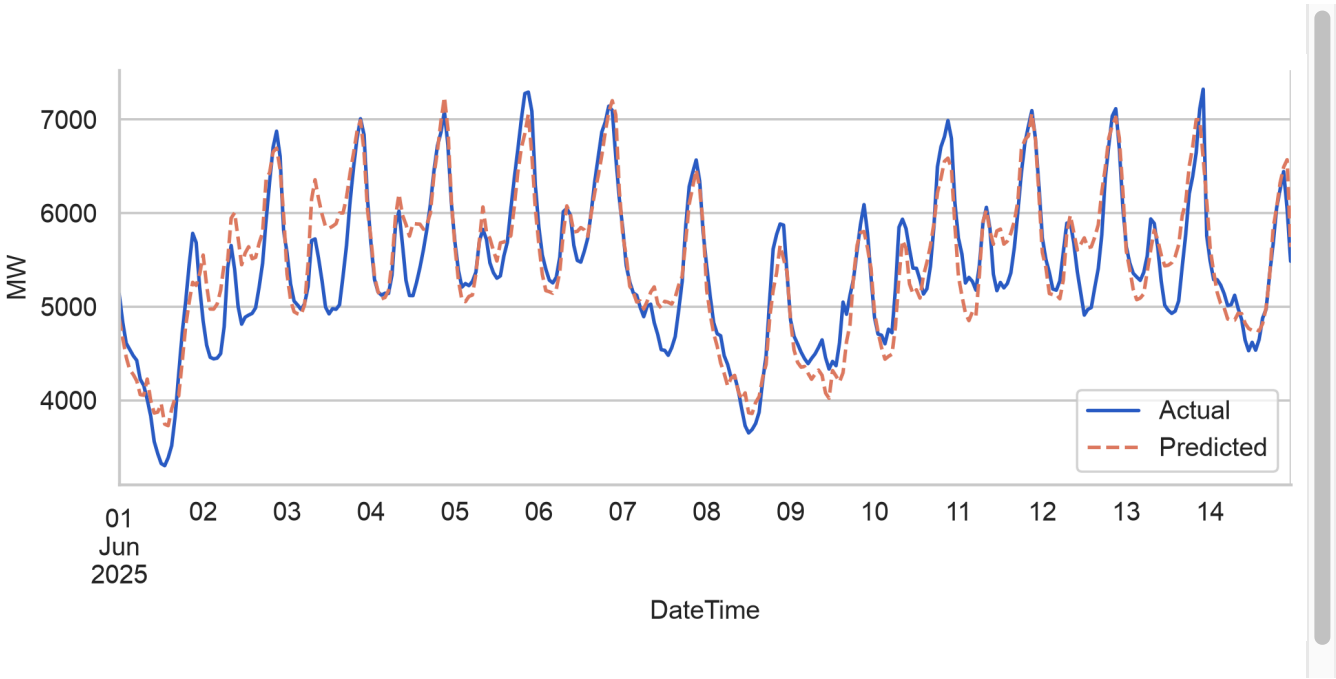
	model	mae_mean	rmse_mean	mape_mean	mape_std
2	LightGBM	201.451715	296.578291	0.032711	0.007269
1	Random Forest	217.293322	331.002114	0.035472	0.008034
0	Linear Regression	246.806452	348.224245	0.039196	0.007359
3	MLP (neural network)	409.488313	533.091900	0.063881	0.052476



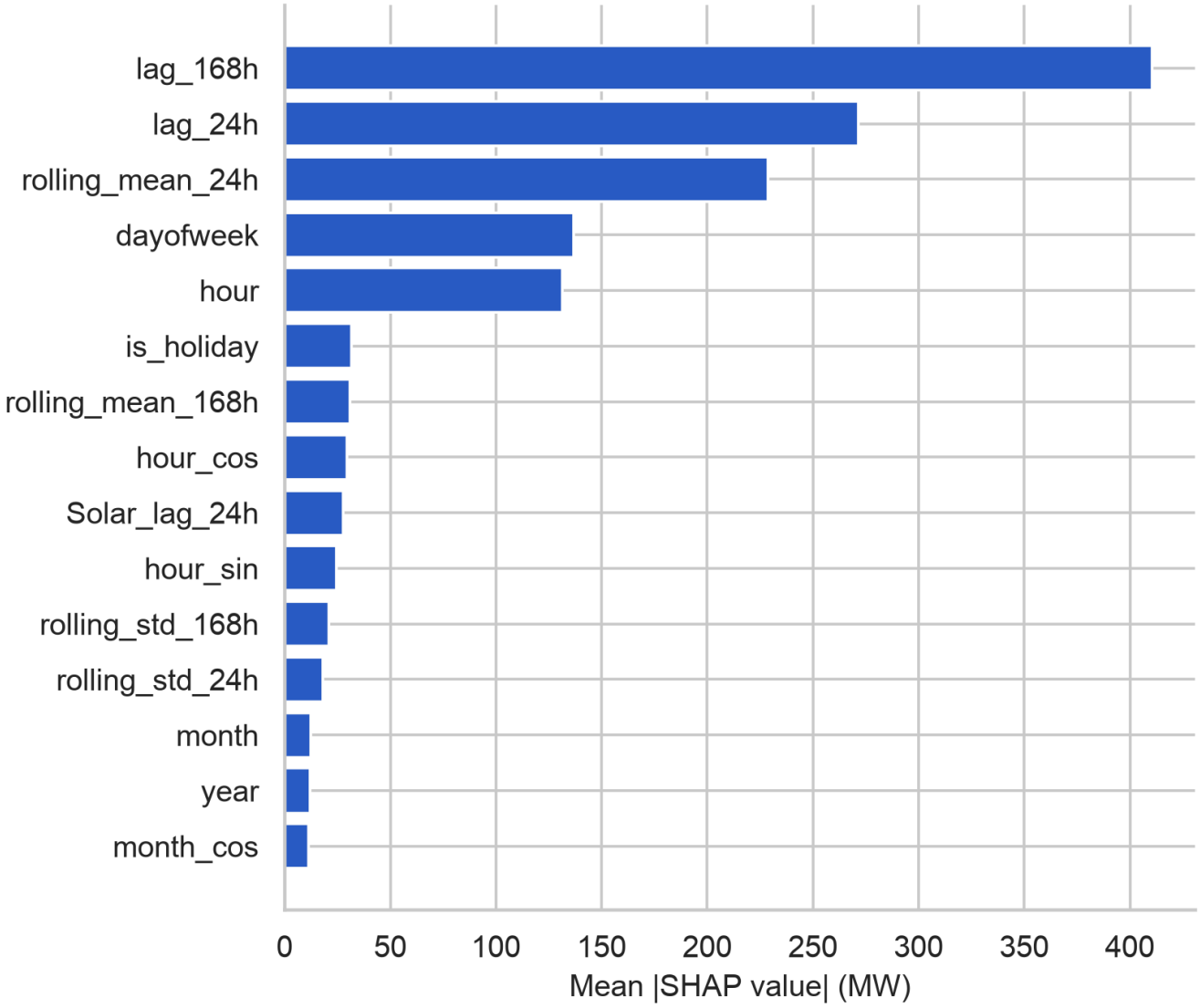
Every model clears the naive baseline by a wide margin, and **LightGBM wins clearly** — including against `MLP (neural network)`, a lightweight feedforward net (`sklearn.neural_network.MLPRegressor`, zero new dependencies) included specifically to test whether a neural approach earns its extra complexity here. It lands close to Random Forest, but neither catches LightGBM — a useful, concrete data point against reaching for a neural network on a tabular, feature-engineered version of this kind of problem.

6 Final chronological holdout

```
{'mae': 258.6064, 'rmse': 424.0605, 'mape': 0.0428}
```



7 Model interpretability



`lag_168h` outranks `lag_24h` here, matching the ACF finding above where the weekly echo was slightly stronger. The genuine surprise: `Solar_lag_24h` **lands in the top 10**, ahead of `is_holiday`. I didn't expect yesterday's solar output to say much about today's demand directly — my best guess is that it's acting as an indirect proxy for season and weather, not that solar generation itself drives consumption. I'd want more than a SHAP ranking alone before treating this as more than a proxy effect.

8 Discussion: limitations and future work

- **This dataset is live and not fully reproducible.** Every number above reflects one snapshot; a fresh download adds rows, so the quoted row counts and test-set contents are tied to this download.
- **Outage gaps require approximate reconstruction.** An 83-hour linear interpolation cannot recover real demand swings. Evaluation touching these gaps uses partly reconstructed observations.

- **Weather is not observed directly.** Calendar features capture seasonal patterns but cannot represent individual temperature changes. Adding weather observations could improve forecasts.
- **Results apply to Romania's grid and the evaluated period.** Changes in demand, generation capacity, or market conditions may require retraining and renewed chronological evaluation.
- **The role of lagged solar output needs further investigation.** Its SHAP ranking suggests predictive value but does not establish a causal relationship with consumption.

9 Conclusion

Romania's hourly consumption-and-production data supports demand forecasting while exposing practical modeling challenges: generation features must be lagged to avoid leakage, and multi-hour outage gaps need to be distinguished from routine DST artifacts.

LightGBM outperformed the regression, Random Forest, and neural-network baselines in this evaluation. The seasonal-decomposition feature added no measurable benefit after leakage checks, while yesterday's solar output ranked highly in the model explanation. These findings support the value of chronological evaluation, explicit baselines, and careful feature engineering, while leaving the solar proxy effect as an open question.